

Inheritance

- When a class derives from another class.
- The child class will inherit all the public and protected properties and methods from the parent class.
- In addition, it can have its own properties and methods.
- An inherited class is defined by using the **extends** keyword.
- **Reference:** [PHP: Object Inheritance - Manual](#)

Parent class:

```
class Vehicle {  
    protected $color;  
    private $speed;  
    public function __construct($color, $speed){  
        $this->color = $color;  
        $this->speed = $speed;  
    }  
    public function getColor(){  
        return $this->color;  
    }  
    public function getSpeed(){  
        return $this->speed;  
    }  
    public function accelerate($amount){  
        $this->speed += $amount;  
        echo "The vehicle accelerates to {$this->speed} km/h.<br>";  
    }  
    public function displayInfo() {  
        echo "Color: {$this->color}<br>";  
        echo "Speed: {$this->speed} km/h<br>";  
    }  
}
```

Child class:

```
class Car extends Vehicle{
    private $brand;
    public function __construct($color,$speed,$brand) {
        parent::__construct($color,$speed);
        $this->brand = $brand;
    }
    public function getbrand(){
        return $this->brand;
    }
    public function displayInfo(){
        parent::displayinfo();
        echo "brand: {$this->brand}<br>";
    }
}
```

Test parent & child class:

//Create instance of the classes

```
$vehicle=new Vehicle("red",50);  
$car=new Car("blue",100,"bmw");
```

//Accessing inherited methods and attributes

```
echo "vehicle color {$vehicle->getColor()}<br>";  
echo "car color {$car->getColor()}<br>";  
echo "car brand {$car->getbrand()}<br>";
```

//Invoking overridden methods

```
echo $vehicle->displayInfo();  
echo $car->displayInfo();
```

//Modifying inherited behavior

```
echo $car->accelerate(30);
```

Abstract

- Abstract classes and methods are when **the parent class has a named method, but need its child class(es) to fill out the tasks.**
- An abstract class is a class that contains at least one abstract method. An abstract method is a method that is **declared**, but **not implemented** in the code.
- An **abstract class** or method is defined with the abstract keyword
- If you declare one or more methods of a class to be abstract, you must also declare the **whole class to be abstract** too.

Abstract

- You can't instantiate an abstract class -> that is can't create an object from it directly
- when you create an abstract class, you are essentially creating a template, rather than a fully fledged class.
- You are saying that any child classes must implement any abstract methods in the abstract class (unless those child classes are themselves declared to be abstract).
- You can mix abstract and non-abstract methods within an abstract class.

Abstract class:

abstract class shape

```
{
    protected $color;
    protected $isfilled;
    public function __construct($color,$isfilled)
    {
        $this->color=$color;
        $this->isfilled=$isfilled;
    }
    public function setColor($color)
    {
        $this->color=$color;
    }
    public function getColor()
    {
        return $this->color;
    }
    public function isfilled()
    {
        return $this->isfilled;
    }
    public function fill()
    {
        $this->isfilled=true;
    }
    abstract public function getArea();
}
```

Subclass: square

```
class square extends shape
{
    private $length;
    public function __construct($col,$isfilled,$length)
    {
        $this->length=$length;
        parent::__construct($col,$isfilled);
    }
    public function setLength($length)
    {
        $this->length=$length;
    }
    public function getLength()
    {
        return $this->length;
    }
    public function getArea()
    {
        return pow($this->length,2);
    }
}
```


Task:

- create subclass circle.
- Define a constructor to add one more parameter radius
- Write functions for get and set the radius
- Write function for get area of the circle

Interfaces

- Interfaces work in a similar way, in that they declare a consistent set of methods that classes must implement.
- However, whereas an abstract class has a parent - child relationship with the class that extends it, **this relationship doesn't exist with interfaces.**
- Interfaces **can't contain properties**; they can only contain method declarations (which **can't contain any implementation code**).
- All methods in an interface **must be public**
- a class can implement **more than one** interface at once

Parent class : shape

```
interface Shape {  
    public function calculateArea();  
    public function calculatePerimeter();  
}
```

Subclass : Circle

```
class Circle implements Shape {  
    private $radius;  
    public function __construct($radius) {  
        $this->radius = $radius;  
    }  
    public function calculateArea() {  
        return M_PI * pow($this->radius, 2);  
    }  
    public function calculatePerimeter() {  
        return 2 * M_PI * $this->radius;  
    }  
}
```

Subclass : Rectangle

```
class Rectangle implements Shape {  
    private $length;  
    private $width;  
    public function __construct($length, $width) {  
        $this->length = $length;  
        $this->width = $width;  
    }  
    public function calculateArea() {  
        return $this->length * $this->width;  
    }  
    public function calculatePerimeter() {  
        return 2 * ($this->length + $this->width);  
    }  
}
```

Interfaces vs. Abstract class

- Interfaces cannot have properties, while abstract classes can
- All interface methods must be public, while abstract class methods is public or protected
- All methods in an interface are abstract, so they cannot be implemented in code and the abstract keyword is not necessary
- Classes can implement an interface while inheriting from another class at the same time

Task1:

- a. Create an abstract Employee class as the base class, which will contain common properties and methods.
 - Properties: firstName, lastName, and salary.
 - A constructor method to initialize the properties.
 - A method called getFullName() that returns the full name of the employee.
 - An abstract method called calculateEarnings() that calculates the earnings for each type of employee.
- b. Create the following subclasses that inherit from the Employee class
 - I. HourlyEmployee:
 - Additional properties: hoursWorked and hourlyRate.
 - The calculateEarnings() method should calculate the earnings based on the hours worked and the hourly rate.
 - II. SalariedEmployee:
 - The calculateEarnings() method should simply return the fixed salary.
 - III. CommissionEmployee:
 - Additional properties: sales and commissionRate.
 - The calculateEarnings() method should calculate the earnings based on the total sales and the commission rate.

Task2:

- a. Create an interface **Movable** which will contain method **move()**
- b. Create a parent class **Vehicle**
 - Properties: **brand**
 - Implement a constructor that takes parameter **brand**
 - A method **startEngine()** that displays a message with the brand of the vehicle
- c. Create the child class **Car** that extends the **Vehicle** class and implement the **Movable** interface
 - Implement a constructor that takes parameter **brand**
 - Implement a method **move()** from movable interface that displays a message indicating the car is moving
- d. Create an instance of the "Car" class and invoke the methods to test the code.

[Task1 answer](#)

[Task2 answer](#)